

EXAMINATION MANAGEMENT SYSTEM â PROJECT DOCUMENTATION

This document explains the Examination Management System (EMS) implemented in this workspace, how it works, system architecture, database design, key components and a step-by-step demo flow you can show to an instructor.

1) QUICK SUMMARY

- * Purpose: a lightweight exam scheduling and administration system that supports student/subject management, exam scheduling, room & bench setup, automated seat allocation, faculty duty assignment, and hall-ticket generation.

- * Technology: MySQL (schema + triggers + stored procedures) and Java Swing (JDK 11+), JDBC.

- * Layers: UI (Swing) â Service â DAO â MySQL.

2) REPOSITORY LAYOUT (IMPORTANT FILES)

- * db/full_workbench_script.sql â full schema, triggers, stored procedures, and sample data.

- * resources/database.properties â JDBC config used in development (overridden by env vars in production via 'DBConfig').

- * Java sources: 'src/com/ems/...':

- * UI: src/com/ems/ui/LoginFrame.java, src/com/ems/ui/DashboardFrame.java, src/com/ems/ui/HallTicketDialog.java

- * Panels:

- src/com/ems/ui/panels/StudentPanel.java, src/com/ems/ui/panels/ExamPanel.java, src/com/ems/ui/panels/RoomBenchPanel.java, src/com/ems/ui/panels/SeatingAllocationPanel.java, src/com/ems/ui/panels/FacultyDutyPanel.java, src/com/ems/ui/panels/SubjectPanel.java, src/com/ems/ui/panels/DepartmentPanel.java

- * Config/DB: src/com/ems/config/DBConfig.java, src/com/ems/config/DBConnection.java

- * DAO/Service: 'src/com/ems/dao/*', 'src/com/ems/service/*' (e.g. 'ExamDAO', 'StudentDAO', 'ExamService')

- * Utilities: src/com/ems/util/UiUtil.java, src/com/ems/util/PasswordUtil.java, src/com/ems/util/LoggerUtil.java

(Click the links above to open files in the editor.)

3) DATABASE DESIGN â IMPORTANT TABLES (HIGH-LEVEL)

- * 'student(usn, name, email, dept_id, semester)' â students master data.
- * 'subject(subject_code, dept_id, subject_name, semester)' â subjects.
- * 'exam(exam_id, exam_date, subject_code, exam_type)' â exam schedule.
- * 'room(room_no, block, total_benches)' and 'bench(bench_no, room_no, capacity)' â physical seating layout.
- * 'seating_allocation(seat_id, bench_no, usn, exam_id, seat_position)' â seat assignments per exam.
- * 'faculty(faculty_id, faculty_name, dept_id, workload, availability)' and 'faculty_duty(...)' â faculty & duties.
- * 'app_user(username, password_hash, role, is_active)' â application users/roles.

Refer to db/full_workbench_script.sql for FKs, CHECKs, triggers and stored procedures.

4) TRIGGERS & STORED PROCEDURES

- * 'trg_prevent_bench_overflow' â prevents bench overflow and seat-position overflow.
- * 'sp_auto_allocate_seats(p_exam_id)' â allocates seats to all students of the subject's department in order.
- * 'sp_assign_faculty_duties(p_exam_id, p_room_no, p_role, p_required_count)' â assigns available faculty up to a workload cap.

These procedures encapsulate the complex allocation logic and should be executed by the application via DAO calls (see 'AllocationDAO', 'FacultyDutyDAO').

5) APPLICATION ARCHITECTURE

- * DB: access via 'DBConnection.getConnection()' which reads 'resources/database.properties' but prefers environment variables ('DB_URL', 'DB_USER', 'DB_PASSWORD', 'DB_DRIVER').
- * DAO layer: low-level SQL + ResultSet â Model mapping. See 'src/com/ems/dao' for examples.
- * Service layer: business validations and workflow orchestration. See 'src/com/ems/service'.
- * UI layer: Swing frames & panels under 'src/com/ems/ui'. 'DashboardFrame' contains navigation and content cards for panels.
- * Utilities: 'UiUtil' (styling + dialogs), 'PasswordUtil' (PBKDF2 password hashing), 'LoggerUtil' (file logging).

6) KEY RUNTIME FLOWS (DETAILED STEP-BY-STEP) â DEMO SCRIPT TO SHOW INSTRUCTOR

Follow this script during the demo. Each step indicates the components involved.

1) Setup environment & DB (one-time)

- * Start MySQL, run 'db/full_workbench_script.sql' in Workbench to create the schema and sample data.

- * Place MySQL Connector/J JAR in 'lib/' (e.g. 'mysql-connector-j-9.7.0.jar').

- * Optionally set environment variables for production:

```
""powershell
$env:DB_URL='jdbc:mysql://localhost:3306/examination_management_system?useSSL=false&serverTimezone=UTC'
$env:DB_USER='root'
$env:DB_PASSWORD='yourpassword'
""
```

2) Start the application (local dev)

- * Compile (from project root):

```
""powershell
Get-ChildItem -Path "src" -Recurse -Filter *.java | ForEach-Object { (Resolve-Path -Relative $_.FullName) } | Set-Content -Path sources.txt
cmd /c "javac -d out @sources.txt"
""
```

- * Run the app (ensure 'lib/mysql-connector-j-*.jar' present):

```
""powershell
java -cp "out;lib/*" com.ems.App
""
```

3) Login (UI)

- * Use seeded accounts (for demo):

- * Admin: 'admin' / 'admin123' (role: ADMIN)

- * Exam Cell: 'examcell' / 'exam123' (role: EXAM_CELL)

- * Viewer: 'viewer' / 'view123' (role: VIEWER)

- * Authentication flow: 'LoginFrame' â†’ 'AuthService' â†’ 'UserDAO.authenticate()' uses 'PasswordUtil.verify()'; legacy plaintext passwords are migrated on first successful login.

4) Student Management (Admin)

- * Add a student: fill 'USN', 'Name', 'Email', 'Dept ID', 'Sem' â†’ 'Add' (validations in 'StudentService' + 'StudentDAO' persists to 'student' table).

- * Update student: enter the 'USN' and new fields â†’ 'Update'.

- * Delete student: enter 'USN' â†’ 'Delete by USN' (prompts confirmation).

- * Generate Hall Ticket: select student row or enter USN â†’ 'Generate Hall Ticket' (fetches exams via 'ExamService.hallTicketForStudent()' â†’ 'ExamDAO.getExamsForStudent()') and opens 'HallTicketDialog' for print/preview).

5) Subject & Exam scheduling

- * Manage subjects in 'Subject Management' (add/update/delete).

- * Create exam in 'Exam Scheduling' with a date picker and subject dropdown (subject list refreshed from DB).

6) Room & Bench Setup

- * Add rooms & benches with capacities. UI will validate numeric inputs.

7) Seat Allocation & Faculty Assignment

- * Run 'Run Auto Allocation' in 'Seat Allocation' (calls 'sp_auto_allocate_seats' via 'AllocationDAO' or 'AllocationService').
- * Auto-assign faculty duties using 'Faculty Assignment' (calls 'sp_assign_faculty_duties').
- * Clear allocations/duties if you want to re-run (UI has 'Clear' actions).

8) Logging & Troubleshooting

- * Errors are logged to 'logs/app.log' (see 'src/com/ems/util/LoggerUtil.java'). The UI shows friendly errors and critical exceptions are recorded with stack traces.

7) WHERE TO FIND IMPORTANT LOGIC

- * Seat allocation: 'src/com/ems/dao/AllocationDAO.java' and 'db/full_workbench_script.sql' ('sp_auto_allocate_seats').
- * Faculty duty assignment: 'src/com/ems/dao/FacultyDutyDAO.java' and 'sp_assign_faculty_duties' in SQL script.
- * Hall ticket: 'src/com/ems/service/ExamService.hallTicketForStudent()' & 'src/com/ems/dao/ExamDAO.getExamsForStudent()' & 'src/com/ems/ui/HallTicketDialog.java'.
- * Authentication/migration: 'src/com/ems/dao/UserDAO.java' and 'src/com/ems/util/PasswordUtil.java'.

8) COMMON INSTRUCTOR Q&A (PREPARED ANSWERS)

Q: How are seats assigned automatically?

A: The stored-procedure 'sp_auto_allocate_seats' computes a list of bench-seat slots and assigns students by row-number ordering; it enforces capacity checks and uses transactions to avoid partial assignments.

Q: How is faculty workload managed?

A: 'sp_assign_faculty_duties' picks available faculty with workload below a cap (SQL cursor) and increments workload on assignment; the UI calls it via 'FacultyDutyService'.

Q: Is authentication secure?

A: Passwords are stored using PBKDF2 hashes via 'PasswordUtil'. Legacy plaintext passwords are migrated on login; in production provide hashed passwords or reset them.

Q: Can the application scale?

A: This is a single-node Swing app backed by MySQL. Scaling to many users requires moving UI to a web UI or desktop clients coordinating via a server API.

9) TESTING & DEMO CHECKLIST (STEP-BY-STEP)

Before your demo, ensure:

1. MySQL running and DB initialized ('db/full_workbench_script.sql').
2. 'lib/mysql-connector-j-*.jar' present.
3. 'out/' classes compiled ('javac -d out @sources.txt').
4. Launch the app and login as 'admin'.

Walkthrough to show instructor:

1. Show DB schema in MySQL Workbench (tables, FKs, triggers).
2. Show 'LoginFrame' and successful login.
3. Add a sample student and refresh the table.
4. Add a subject and create an exam for that subject.
5. Setup a room and bench with capacity.
6. Run 'Run Auto Allocation' for the exam and show seating allocation results.
7. Run 'Auto Assign Faculty' and show assigned duties.
8. Select the student and 'Generate Hall Ticket' â preview and print.

10) KNOWN LIMITATIONS AND NEXT IMPROVEMENTS

- * No build system (Maven/Gradle); adding one would simplify dependency management and packaging.
- * Basic UI â accessibility and UX can be improved (keyboard navigation, form autofill on row select).
- * No automated tests â add unit + integration tests for DAOs and services.
- * No RBAC beyond simple roles; add permissions checks in the UI and service layer for stronger access control.
- * Consider moving to a server-client architecture for multi-user access and concurrency control.

If you'd like, I can:

- * Produce a one-page slide (PDF) summarizing the demo steps only.
- * Generate a shorter "demo script" version tailored to a 10-minute demonstration.
- * Add inline comments in the most important files to help present code during the walkthrough.

Which of the above would help you most for the instructor presentation?